

Week 1, Day 1: Medical Data Structures - Building Clinical Intuition

Core Insight: Your Clinical Mind Already Thinks in Data Structures

As a clinician, you already organize information systematically - patient histories, differential diagnoses, treatment protocols. Python data structures are just formal ways to represent these mental models.

Today's Goal (30-45 minutes)

Understand the three essential Python data structures you'll use daily in medical ML, connecting each to clinical workflows you know intimately.

Part 1: Lists - Sequential Clinical Data (10 minutes)

Clinical Context: Vital Signs Over Time

```
python

# This is how you naturally think about vitals
patient_temps = [98.6, 99.2, 101.4, 102.1, 100.8, 99.9]
bp_systolic = [120, 125, 140, 155, 145, 130]

# Clinical questions you ask naturally:
print(f"Peak temperature: {max(patient_temps)}°F")
print(f"Temperature trend: {patient_temps[-1] - patient_temps[0]}°F change")

# Is patient improving? (Last 3 readings vs first 3)
recent_temps = patient_temps[-3:]
initial_temps = patient_temps[:3]
print(f"Recent average: {sum(recent_temps)/len(recent_temps):.1f}")
print(f"Initial average: {sum(initial_temps)/len(initial_temps):.1f}")
```

Critical Insight

You already think in sequences - symptom progression, medication timelines, lab trends. Lists just formalize this thinking.

Practice Exercise

Create a list of troponin values and write code to:

1. Find the peak value
 2. Determine if trending up or down
 3. Check if any value exceeds normal threshold (0.04 ng/mL)
-

Part 2: Dictionaries - Patient Records (15 minutes)

Clinical Context: Structured Patient Data

python

This mirrors how you mentally organize a patient

```
patient_record = {
    "mrn": "ED240830001",
    "age": 67,
    "chief_complaint": "chest pain",
    "vitals": {
        "temp": 98.9,
        "bp": [140, 88],
        "hr": 95,
        "spo2": 97
    },
    "labs": {
        "troponin": 0.12,
        "bnp": 450,
        "creatinine": 1.2
    },
    "allergies": ["penicillin", "iodine"],
    "medications": ["metoprolol", "atorvastatin"]
}
```

Clinical reasoning in code:

```
def assess_mi_risk(patient):
    """Calculate MI risk based on clinical factors"""
    risk_score = 0

    # Age factor
    if patient["age"] > 65:
        risk_score += 2

    # Troponin elevation
    if patient["labs"]["troponin"] > 0.04:
        risk_score += 3

    # Chest pain presentation
    if "chest pain" in patient["chief_complaint"].lower():
        risk_score += 2

    return risk_score

risk = assess_mi_risk(patient_record)
print(f"MI Risk Score: {risk}/7")
```

Why Dictionaries are Powerful for Medical Data

- **Labeled access:** `patient["labs"]["troponin"]` is clearer than `patient[5][2]`
- **Flexible structure:** Easy to add new fields without breaking existing code
- **Natural mapping:** Mirrors how you think about patient attributes

Critical Practice

Build a patient assessment function that:

1. Checks for drug allergies before prescribing
 2. Calculates eGFR from creatinine and age
 3. Flags critical values (troponin >0.4, HR >120, etc.)
-

Part 3: Sets - Clinical Decision Support (10 minutes)

Clinical Context: Contraindications and Interactions

```
python
```

```
# Patient's current medications
```

```
current_meds = {"warfarin", "metoprolol", "lisinopril", "atorvastatin"}
```

```
# Medications you want to prescribe
```

```
proposed_meds = {"aspirin", "clopidogrel", "metoprolol"}
```

```
# Critical safety checks
```

```
warfarin_interactions = {"aspirin", "clopidogrel", "heparin"}
```

```
# Check for dangerous interactions
```

```
dangerous_combo = current_meds & warfarin_interactions
```

```
if dangerous_combo:
```

```
    print(f"⚠️ WARNING: Bleeding risk with {dangerous_combo}")
```

```
# Find medication overlaps
```

```
duplicates = current_meds & proposed_meds
```

```
if duplicates:
```

```
    print(f"Duplicate medications: {duplicates}")
```

```
# What's actually new?
```

```
new_medications = proposed_meds - current_meds
```

```
print(f"New prescriptions needed: {new_medications}")
```

Clinical Power of Sets

- **Intersection** ($\cap$): Find overlaps (drug interactions, comorbidities)
 - **Difference** ($\setminus$): What's missing or new
 - **Union** ($\cup$): Combined medication list
 - **Fast membership testing**: `"penicillin" in allergies`
-

Part 4: Real Clinical Scenario (10 minutes)

Putting It All Together: ED Triage System

python

```
def triage_patient(patient_data):  
    """Simplified triage based on clinical parameters"""  
  
    # Extract key data (dictionary access)  
    vitals = patient_data["vitals"]  
    chief_complaint = patient_data["chief_complaint"].lower()  
  
    # Critical vital thresholds (using comparisons)  
    critical_signs = []  
    if vitals["bp"][0] > 180: # Systolic BP  
        critical_signs.append("hypertensive")  
    if vitals["hr"] > 120:  
        critical_signs.append("tachycardic")  
    if vitals["temp"] > 101.5:  
        critical_signs.append("febrile")  
  
    # High-risk chief complaints (set membership)  
    high_risk_complaints = {  
        "chest pain", "shortness of breath", "stroke symptoms",  
        "severe abdominal pain", "altered mental status"  
    }  
  
    # Triage decision logic  
    if critical_signs or any(complaint in chief_complaint for complaint in high_risk_complaints):  
        return "ESI 2 - Urgent"  
    else:  
        return "ESI 3 - Less Urgent"  
  
    # Test with your patient  
    result = triage_patient(patient_record)  
    print(f"Triage Decision: {result}")
```

Today's Key Realizations

1. You Already Think in Data Structures

Your clinical reasoning naturally organizes data:

- **Sequential:** Vital trends, medication schedules
- **Hierarchical:** Systems → organs → lab values
- **Categorical:** Differential diagnoses, treatment options

2. Medical Data is Messy - Python Helps

```
python

# Handle missing or invalid data
def safe_get_lab(patient, lab_name, default=None):
    try:
        return patient["labs"][lab_name]
    except KeyError:
        return default

troponin = safe_get_lab(patient_record, "troponin", "not_drawn")
```

3. Code Should Mirror Clinical Thinking

Good medical code reads like clinical reasoning - clear, systematic, defensive.

Tomorrow's Preview: List Comprehensions

You'll learn to write efficient filters like:

```
python

# Find all hypertensive patients
hypertensive = [p for p in patients if p["vitals"]["bp"][0] > 140]

# Critical troponin values
elevated_trop = [p for p in patients if p["labs"].get("troponin", 0) > 0.04]
```

Critical Self-Assessment Questions

1. **Can I explain each data structure to a colleague?**
2. **Do I see how this connects to my ED workflow system?**
3. **What clinical scenarios would benefit from each structure?**
4. **Where in my existing code are these patterns already present?**

Time Check: If you spent more than 45 minutes today, that's normal - you're building foundational understanding. Better to go deep on basics than rush through concepts superficially.